
EDA Report — Automobile Dataset

1. Introduction

Description of the Dataset

The automobile dataset contains specifications and pricing for a range of car models from various manufacturers. Each row represents a single car model with attributes covering its physical dimensions, engine specifications, performance metrics, and retail price.

Column	Description
make	Manufacturer / brand name
fuel-type	Type of fuel — gas or diesel
body-style	Vehicle shape — sedan, hatchback, wagon, hardtop, convertible
engine-size	Engine displacement in cubic centimetres (cc)
horsepower	Engine output measured in horsepower
city-mpg / highway-mpg	Fuel economy in miles per gallon
price	Retail price in USD

Key facts about the raw dataset:

- 205 rows and 26 columns before cleaning
- 21 unique manufacturers
- 5 body style categories
- Missing values were stored as `?` strings throughout the file rather than true null values

2. Data Cleaning

Summary of Methods

The following steps were taken to clean the dataset before any analysis was performed.

Step 1 — Remove redundant columns

Two columns were dropped: `symboling` (an insurance risk rating) and `normalized-losses` (an insurance loss metric). Neither column was relevant to the analysis questions being explored.

```
automobiles_df = automobiles_df.drop(columns=['normalized-losses',  
                                             'symboling'])
```

Step 2 — Remove duplicate rows

`drop_duplicates()` was called to check for and remove any fully repeated rows. No duplicates were found in this dataset.

```
automobiles_df = automobiles_df.drop_duplicates()
```

Step 3 — Handle missing values

The dataset used `?` as a placeholder for missing data rather than leaving entries blank. These were first replaced with `NaN` so pandas could recognise them, then all rows containing any missing value were dropped.

```
automobiles_df = automobiles_df.replace('?', np.nan)  
automobiles_df = automobiles_df.dropna()
```

This reduced the dataset from 205 rows to 193 rows — a loss of 12 rows (less than 6%).

Step 4 — Fix data types

Because of the mixed `?` values, several numeric columns were read in as strings (`object` type). After cleaning, these were explicitly converted to their correct types using `astype()`.

```
automobiles_df[int_cols] = automobiles_df[int_cols].astype(np.int64)  
automobiles_df[float_cols] = automobiles_df[float_cols].astype(float)
```

Visualisations During Cleaning

- The `head()` output after dropping columns confirmed the correct columns were removed
 - The `dtypes` output after type conversion confirmed all columns were in their correct numeric format
-

3. Missing Data

Was there missing data?

Yes. After converting `?` to `NaN`, 7 columns were found to have missing values:

Column	Missing Count	Missing %
normalized-losses	41	20.0%
bore	4	2.0%
stroke	4	2.0%
price	4	2.0%
horsepower	2	1.0%
peak-rpm	2	1.0%
num-of-doors	2	1.0%

How was it handled?

normalized-losses was removed entirely as a column. With 20% of rows missing this value, imputing it would have introduced significant bias and the column was not going to be used in any of the analysis anyway.

All other columns — the affected rows were dropped using `dropna()`. Only 12 rows were removed in total which is less than 6% of the dataset. Dropping these rows was the safest option as it keeps the data honest rather than guessing replacement values for things like price or horsepower which are difficult to estimate reliably.

There were no unexplained or random gaps in the data. The missing entries most likely represent car models where certain specifications were simply never recorded in the original source.

4. Data Stories and Visualisations

```
In [11]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
%matplotlib inline

automobiles_df = pd.read_csv(r'C:\Users\bigma\Downloads\JS26010019557\Level 2 - Dat
automobiles_df = automobiles_df.drop(columns=['normalized-losses', 'symboling'])
automobiles_df = automobiles_df.drop_duplicates()
automobiles_df = automobiles_df.replace('?', np.nan)
automobiles_df = automobiles_df.dropna()
```

```
int_cols = ['engine-size', 'curb-weight', 'horsepower', 'peak-rpm',
            'city-mpg', 'highway-mpg', 'price']
automobiles_df[int_cols] = automobiles_df[int_cols].astype(np.int64)

float_cols = ['wheel-base', 'length', 'width', 'height', 'bore', 'stroke', 'compression-ratio']
automobiles_df[float_cols] = automobiles_df[float_cols].astype(float)
```

Story 1 — The Most Expensive Cars Are Powerful but Fuel-Hungry

Visualisation: Top 5 Most Expensive Cars — price bar chart and city vs highway MPG grouped bar chart

The 5 most expensive cars in the dataset are all European luxury brands — Mercedes-Benz (\$40,960–\$45,400), BMW (\$36,880–\$41,315), and Porsche (\$37,028). Looking at the MPG chart alongside the prices makes the trade-off very clear. The Mercedes models only get 14 city / 16 highway mpg which are among the worst fuel economy figures in the entire dataset. The Porsche manages 17/25 mpg which is slightly better but still well below average.

Assumption: Buyers of these cars are paying for performance and prestige, not practicality. The high price is driven by engine power and brand reputation rather than any kind of efficiency. These are not cars people choose to save money on fuel.

```
In [12]: top5_expensive = automobiles_df.nlargest(5, 'price')[['make', 'body-style', 'price']]

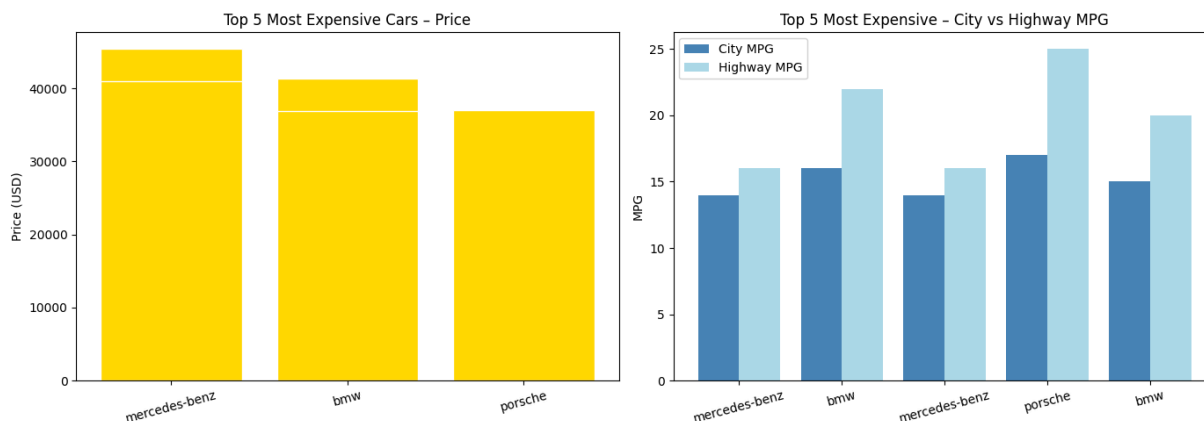
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

labels = [row['make'] for _, row in top5_expensive.iterrows()]

# Bar chart for price
axes[0].bar(labels, top5_expensive['price'], color='gold', edgecolor='white')
axes[0].set_title('Top 5 Most Expensive Cars - Price')
axes[0].set_ylabel('Price (USD)')
axes[0].tick_params(axis='x', rotation=15)

# Grouped bar chart for city vs highway MPG
x = range(len(top5_expensive))
axes[1].bar([i - 0.2 for i in x], top5_expensive['city-mpg'], width=0.4, label='City')
axes[1].bar([i + 0.2 for i in x], top5_expensive['highway-mpg'], width=0.4, label='Highway')
axes[1].set_title('Top 5 Most Expensive - City vs Highway MPG')
axes[1].set_ylabel('MPG')
axes[1].set_xticks(list(x))
axes[1].set_xticklabels(labels, rotation=15)
axes[1].legend()

plt.tight_layout()
plt.show()
```



Story 2 — Honda and Volkswagen Are the Most Consistently Fuel Efficient

Visualisation: Average Highway MPG by Manufacturer bar chart

Chevrolet tops the chart at 46.3 mpg average, but this is misleading — it only has 3 very small economy models in the dataset so the average is not representative. Among manufacturers with a sizeable number of models, Honda (13 models, ~35.5 mpg avg) and Volkswagen (~34.9 mpg avg) are the most consistently above the dataset average line. The luxury brands like Mercedes-Benz and Jaguar sit well below the average, reinforcing the pattern from Story 1.

Assumption: This aligns with what these brands have historically been known for. Honda and VW built their reputations on reliable, economical everyday cars. The data confirms that the premium segment almost entirely sacrifices fuel efficiency for performance.

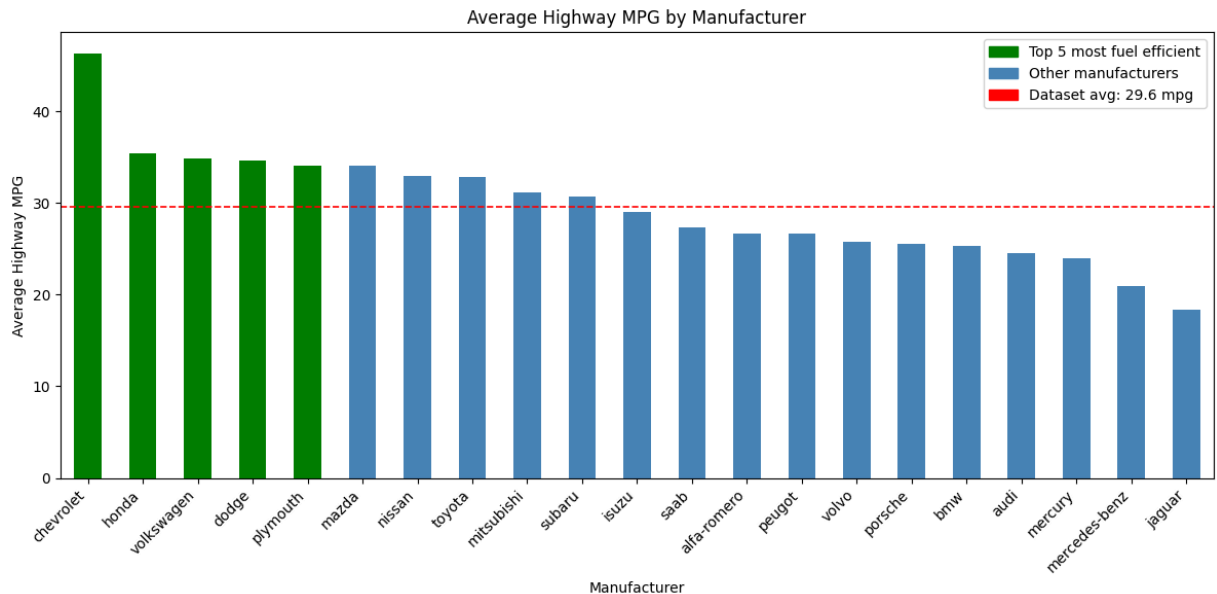
```
In [13]: avg_mpg = automobiles_df.groupby('make')['highway-mpg'].mean().sort_values(ascending=True)

colors = ['green' if i < 5 else 'steelblue' for i in range(len(avg_mpg))]

plt.figure(figsize=(12, 6))
avg_mpg.plot(kind='bar', color=colors)
plt.axhline(avg_mpg.mean(), color='red', linestyle='--', linewidth=1.2)
plt.title('Average Highway MPG by Manufacturer')
plt.xlabel('Manufacturer')
plt.ylabel('Average Highway MPG')
plt.xticks(rotation=45, ha='right')

top5_patch = mpatches.Patch(color='green', label='Top 5 most fuel efficient')
other_patch = mpatches.Patch(color='steelblue', label='Other manufacturers')
avg_line = mpatches.Patch(color='red', label=f'Dataset avg: {avg_mpg.mean()}')
plt.legend(handles=[top5_patch, other_patch, avg_line])

plt.tight_layout()
plt.show()
```



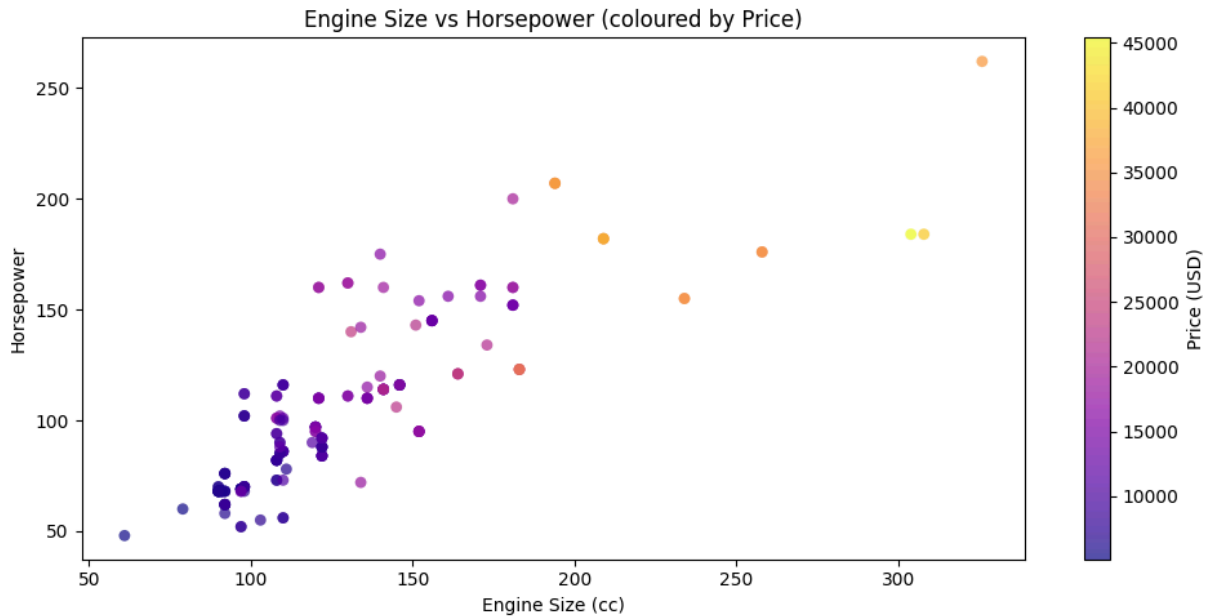
Story 3 — Bigger Engine Means More Power and Always a Higher Price

Visualisation: Engine Size vs Horsepower scatter plot (coloured by price)

The scatter plot shows a clear positive trend — as engine size increases, horsepower increases with it. The price colour coding (plasma colourmap) shows that the most expensive cars (yellow/bright dots) are all clustered in the upper right of the chart where engine size and horsepower are highest. Jaguar has the largest engine at 326cc with 262 horsepower, closely followed by Mercedes-Benz at 304–308cc.

Assumption: Engine size is one of the primary drivers of both performance and price in this dataset. The three variables — engine size, horsepower, and price — all move in the same direction, which makes sense. When buying a premium car you are essentially paying for the engine first and everything else second.

```
In [14]: plt.figure(figsize=(10, 5))
scatter = plt.scatter(automobiles_df['engine-size'], automobiles_df['horsepower'],
                      c=automobiles_df['price'], cmap='plasma', alpha=0.7, edgecolor='black')
cbar = plt.colorbar(scatter)
cbar.set_label('Price (USD)')
plt.title('Engine Size vs Horsepower (coloured by Price)')
plt.xlabel('Engine Size (cc)')
plt.ylabel('Horsepower')
plt.tight_layout()
plt.show()
```



Story 4 — Toyota Dominates the Market and Sedans Rule the Road

Visualisation: Number of Models per Manufacturer bar chart and Body Style Distribution pie chart

Toyota has 32 models in the dataset — nearly double Nissan which is in second place with 18. The top 5 manufacturers by model count are all Japanese brands. For body styles, sedans (47.7%) and hatchbacks (32.6%) together account for over 80% of the entire dataset. Convertibles and hardtops combined make up less than 10%.

Assumption: The dominance of Japanese manufacturers is shown in this dataset. The sedan and hatchback dominance simply reflects what the majority of consumers actually buy. Convertibles and hardtops are niche products and their small representation in the data is exactly what you would expect.

```
In [15]: models_per_make = automobiles_df.groupby('make').size().sort_values(ascending=False)

fig, axes = plt.subplots(1, 2, figsize=(16, 5))

bar_colors = ['red'] + ['steelblue'] * (len(models_per_make) - 1)
models_per_make.plot(kind='bar', ax=axes[0], color=bar_colors)
axes[0].set_title('Number of Models per Manufacturer')
axes[0].set_xlabel('Manufacturer')
axes[0].set_ylabel('Number of Models')
axes[0].tick_params(axis='x', labelsiz=8, rotation=45)

body_style_counts = automobiles_df['body-style'].value_counts()
axes[1].pie(body_style_counts.values, labels=body_style_counts.index, autopct='%1.1f%%',
            startangle=140, colors=sns.color_palette('pastel', len(body_style_counts)))
axes[1].set_title('Body Style Distribution')
```

```
plt.tight_layout()  
plt.show()
```

